

Laboratory Notes

Week 3 Lab: Introduction to Turtlebot3 and Gazebo

1 Aim

This laboratory aims to introduce the student to the Turtlebot3 robot package. It also introduces the Gazebo simulation package that will be used extensively throughout future labs in this subject. This laboratory will cover the following specific areas:

- Introduce the Turtlebot3 (TB3).
- Introduce the TB3 eManual.
- Introduce the Gazebo robotics simulation tool.
- Begin Python programming.
- Publish and subscribe to some ROS topics.
- Write a launch file.

2 Background

This lab explores fundamental concepts in robotics simulation and ROS programming, using the TurtleBot3 platform. We will gain hands-on experience with Gazebo, a physics-based simulator for testing robotic systems in virtual environments, and RViz, a visualisation tool for monitoring sensor data and robot states in real-time. The lab progresses from basic simulations, and robot interaction via teleoperation, to creating ROS packages, developing Python scripts (that implement publisher and subscriber nodes) and writing launch files to automate the execution of multiple ROS components. Large parts of this lab are ‘building blocks’ for the labs that will follow throughout the rest of the session.

3 Procedure

3.1 Introduction to Turtlebot3 and Gazebo

In this section, you will learn the commands to launch several Turtlebot-based Gazebo worlds. Do not launch multiple worlds at once though. Close each world down from the terminal window before launching a new world.

Please note: Gazebo is a resource intensive program. Please allow it sufficient time to boot and shutdown correctly. When loading a new world for the first time, it can take a minute or two depending on the complexity of the world.

3.1.1 Launching Gazebo Worlds

Try launching each of the following worlds, using the given commands (one at a time):

```
1 ros2 launch turtlebot3_gazebo empty_world.launch.py
2 ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
3 ros2 launch ecte477 blocked_road.launch.xml
```

3.1.2 Launch Teleoperation

Now that the worlds have been launched, it is possible to control the turtlebots using teleoperation commands (similar to the turtlesim in the Week 2 Lab). The command for launching teleoperation can be found in the TB3 eManual under Section 6.1.3. The eManual can be found with a quick google search (when you go to Section 6.1.3 make sure that you have “Jazzy” selected at the top of the page).

3.1.3 Resetting the Simulation

In the Week 2 lab you were able to reset the turtlesim simulator via a service call. With simulations in Gazebo it is a little more complicated to reset a simulation. The easiest way is via the ‘Reset the simulation’ button within the Gazebo simulator (it is located at the bottom of the simulation window, near the play button). However, resetting the simulation within Gazebo does not automatically respawn the turtlebot itself. For these reasons, it is often best to simply close the simulation and run the launch file again, whenever you need to reset what you are simulating.

3.1.4 Control the velocity of the Turtlebot from the Command Line

As with the turtlesim simulator, it is possible to control the velocity of the TB3 from the command line. Try this now, using the Week 2 Lab notes as a guide to determine what the correct command is.

3.2 ROS Visualisation with RViz

With only the blocked_road world running, open a new terminal and enter the following command:

```
1 ros2 launch turtlebot3_bringup rviz2.launch.py
```

Open the “Views” panel, via the “Panels” menu at the top of RViz. Once open, choose “ThirdPersonFollower” via the Views Panel “Type” menu – then click the “Zero” button. Now close the Views Panel.

3.2.1 Change the size of the LaserScan indicators

In the “Displays” Panel, find the LaserScan information and change “Size (m)” to 0.1 – what changes can you observe with the LaserScan?

3.2.2 Activate the Camera Topic

At the top of the “Displays” Panel click the “Add” button. Choose the “By Topic” tab in the menu that opens. Now, select ‘/camera/image_raw/Camera’ and press “OK”.

Drive the TB3 around using teleoperation, keeping an eye on the acquired data. These exercises are all for familiarisation with both the Gazebo simulation and RViz – once you have explored a bit, you can close all running ROS programs (via the appropriate terminals).

3.3 Create a ROS Package

This section provides a very basic, step-by-step guide to making ROS2 (python) packages. These steps will be required in later labs. Some specific notes:

- “colcon build” is the command for the build tool in ROS2 and must always be run from the root directory of your workspace (in our case “ros2_ws”). This is very important, as running colcon build from other directories can cause conflicts in your ROS system.
- It is not actually necessary to run “colcon build” before we create the package. It is good practice though, in order to ensure there are no other unbuilt packages before you create your new one.
- “ros2 pkg create” must be run inside the “src” directory.
- In the steps below “jeffs_pkg” is the name of the new package – please change this (from jeff) to your own name ☺

The Steps:

```
1 cd
2 cd ros2_ws
3 colcon build
4 cd src
5 ros2 pkg create --dependencies rclpy --build-type ament_python jeffs_pkg
6 cd ..
7 colcon build
8 cd src/jeffs_pkg
```

3.4 Write a simple python script – ROS Publisher

The goal of this task will be to write a simple script that commands the TB3 to drive in a circular motion. You will be given code that will drive the TB3 forward. You will then need to add some minor additional code to make it move in a circle.

3.4.1 Create a File

Open “Visual Studio Code”

Visual Studio Code should already be set to use the “src” directory inside your workspace as the working directory. However, if not, you should use the “Open Folder” option in Visual Studio Code and select the “src” directory. Using the Explorer in VS Code find the directory of your new package (i.e. jeffs_pkg) and click on it. You should now see a directory inside the package with exactly the same name as the package directory (i.e. in my case: jeffs_pkg/jeffs_pkg). Right click on this directory and choose “New File”. You should call your new file: move_tb3_circle.py

Your new file should now be open in the VS Code editor.

3.4.2 Populate the File

Enter the following code into your new file. You should type the code directly, rather than trying to copy and paste it, as errors can arise from copying and pasting (with characters not always matching).

```

1 import rclpy
2 from rclpy.node import Node
3 from geometry_msgs.msg import TwistStamped
4
5 class MoveTurtlebot(Node):
6
7     def __init__(self):
8         super().__init__('move_turtlebot3_node')
9         self.pub = self.create_publisher(TwistStamped, '/cmd_vel', 1)
10        self.time_passed = 0
11        timer_period = 0.5
12        self.move = TwistStamped()
13        self.timer = self.create_timer(timer_period, self.timer_callback)
14
15    def timer_callback(self):
16        if self.time_passed < 5:
17            self.move.twist.linear.x = 0.5
18        else:
19            self.move.twist.linear.x = 0.0
20        self.pub.publish(self.move)
21        self.time_passed += 0.5
22
23    def main(args=None):
24
25        rclpy.init(args=args)
26        move_turtlebot = MoveTurtlebot()
27        rclpy.spin(move_turtlebot)
28
29    if __name__ == '__main__':
30
31        main()

```

3.4.3 Build the Package

Before you can run your new file, you need to build the package again. However, there are a couple of configuration edits that need to be performed first:

- In VS Code, open the “package.xml” file for your package (located in the package directory). You need to ensure that all of the modules that are imported into your python script are also included in the package dependencies – rclpy should already be included, as we specified this when we created the package. Locate where this is in the “package.xml” file and then add the dependency for geometry_msgs directly below it:

```
1 <depend>rclpy</depend>
2 <depend>geometry_msgs</depend>
```

- Again in VS Code, open the “setup.py” file for your package (located in the package directory). Towards the end of the file there is a setup variable called “entry_points”, you need to edit that section of the file to match that shown below (note that you need to change “jeffs_pkg” to the name of your own package):

```
1 entry_points={
2     'console_scripts': [
3         'move_tb3_circle = jeffs_pkg.move_tb3_circle:main',
4     ],
5 }
```

- Now navigate to the root directory of your workspace (“ros2_ws”) and run “colcon build” again.

3.4.4 Test the file

- Launch Gazebo with empty_world
- Now, use the “ros2 run” command to run your file.

```
1 ros2 run jeffs_pkg move_tb3_circle
```

- The TB3 should drive forwards for 5 seconds and then stop.

3.4.5 Edit the Code

Look through the existing code for your file and try to understand the functionality. Note this has been written using Python’s object oriented approach. Now edit the code so that the TB3 will drive in a circular motion and stop after the allotted time. This should only require two additional lines of code. Try also changing the drive time from 5 seconds to 15 seconds and run the file again. Note that you will need to re-build the package after each edit.

3.5 Write a Simple python script – ROS Subscriber

In a manner similar to Section 3.4, we will now write a script that subscribes to the LIDAR, reads a distance value, and displays this value on screen. Use VS Code again to create and edit the new file, which we will call: lidar_sub.py

The complete code to subscribe to the LIDAR data is given below. Once you have entered the code, you will need to re-build your package again. Just like in Section 3.4, make sure that you add any new dependencies to the “package.xml” file and add an entry point for your script in the “setup.py” file. You can use Section 3.4 as a guide to determine the correct code to be added to these two configuration files.

Once the package is built, launch Gazebo with the blocked_road world and run your file. Check how far from the wall the TB3’s starting position is.

```

1 import rclpy
2 from rclpy.node import Node
3 from sensor_msgs.msg import LaserScan
4
5 class LidarSubscriber(Node):
6
7     def __init__(self):
8         super().__init__('lidar_subscriber')
9         self.subscription = self.create_subscription(LaserScan, '/scan', self.lidar_callback, 5)
10        self.subscription
11
12    def lidar_callback(self, msg):
13        print(msg.ranges[0])
14
15 def main(args=None):
16     rclpy.init(args=args)
17     lidar_subscriber = LidarSubscriber()
18     rclpy.spin(lidar_subscriber)
19     lidar_subscriber.destroy_node()
20     rclpy.shutdown
21
22 if __name__ == '__main__':
23     main()

```

3.6 Write a Launch File

We will now write a simple launch file to launch the move_tb3_circle code that we wrote in Section 3.4.

Steps required:

- Make a new directory within your package called: launch
- Use VS Code to make a new file inside your launch folder called: move_tb3_circle.launch.xml

- Modify the example launch file code shown below so that you are able to correctly launch your move_tb3_circle node.

```

1      <launch>
2          <node name="*****" pkg="*****" exec="*****" output="*****"/>
3      </launch>

```

- Save your launch file and re-build the package. Before rebuilding you will need to edit “setup.py” file again so that the build system is aware that your package contains launch files. This is achieved via editing the setup variable called “data_files”. Edit this section of “setup.py” to match that shown below.

```

1      data_files=[
2          ('share/ament_index/resource_index/packages',
3           ['resource/' + package_name]),
4          ('share/' + package_name, ['package.xml']),
5          (os.path.join('share', package_name, 'launch'), glob(os.path.join('launch', '*launch.[pxy][yma]*'))),
6      ],

```

- You will also need to add the following import statements at the top of your setup.py file:

```

1      import os
2      from glob import glob

```

- Test your launch code, making sure that you already have the gazebo simulator running with an appropriate world.

4 Summary

In today’s laboratory we have explored using Gazebo and RViz with Turtlebot3 models. We also wrote our first python scripts to develop publisher and subscriber nodes in ROS. Finally, we wrote a launch file to launch our new node(s). The concepts covered today will be used throughout the rest of the session, so it is important to take the time to ensure that each concept is well understood. Next week we will look further at the concepts of Publisher and Subscribers in ROS.